Minitest and Routing

The primary method that Rails/Minitest uses for route testing is assert_routing. Here are the Minitest versions of the same standard routes:

minitest/02/test/controllers/projects_controller_test.rb

```
require "test_helper"

class ProjectsControllerTest < ActionController::TestCase
  test "routing" do
    assert_routing "/projects", controller: "projects", action: "index"
    assert_routing({path: "/projects", method: "post"},
      controller: "projects", action: "create")
    assert_routing "/projects/new", controller: "projects", action: "new"
    assert_routing "/projects/1", controller: "projects",
                                  action: "show", id: "1"
    assert_routing "/projects/1/edit", controller: "projects",
                                       action: "edit", id: "1"
    assert_routing({path: "/projects/1", method: "patch"},
      controller: "projects", action: "update", id: "1")
    assert_routing({path: "/projects/1", method: "delete"},
      controller: "projects", action: "destroy", id: "1")
  end
end
```

The first argument to assert_routing represents the path. As you can see from the preceding examples, this argument is either a string representing the URL, or a hash with a :path key representing the URL and a :method key representing the HTTP method being invoked. If no method is specified, the default is GET.

The second argument is a hash representing any of the elements of the route after it is processed by the Rails router, meaning you would expect this argument to specify a :controller, an :action, and any other symbols defined by the route. This second argument does not contain any elements that are expected to be part of the query string as opposed to the base URL.

The third argument is defined as defaults. As far as I can tell, it's essentially merged into the second argument. (The documentation says this parameter is unused, though it's clearly referenced in the source code.) It seems to be safe to leave it as an empty hash if you need the fourth argument. That fourth argument is where you specify key/value pairs for any part of the route you expect to be in the query string.

The assert_routing method validates the routing in both directions. You can also run the routing tests in one direction using assert_generates to go from a string to a hash, and assert_recognizes to go from a hash to a string.